

Programacion Orientada a Objetos

Practica 13

Alumno: Issac Martin Recendiz Hernandez.

Semana 13

**Profesor: Jose Jesus
Collazo**

14 de mayo del 2026

PRÁCTICA SEMANA 13

“Sistema de Empleados y Roles Empresariales”

INTRODUCCIÓN

En esta práctica se desarrolló un sistema de empleados utilizando Programación Orientada a Objetos en C++. El propósito principal fue aplicar temas importantes vistos durante el curso, como las clases abstractas, las interfaces, la herencia y el polimorfismo. Todos estos conceptos ayudan a crear programas mucho más organizados, reutilizables y fáciles de mantener cuando el sistema comienza a crecer.

Actualmente muchas empresas utilizan sistemas que manejan distintos tipos de trabajadores. Aunque todos son empleados, cada uno realiza actividades diferentes dependiendo de su área. Por ejemplo, un programador desarrolla código, un administrador de red monitorea servidores y un técnico de soporte atiende problemas técnicos. Gracias a la programación orientada a objetos es posible representar cada uno de estos roles mediante clases diferentes, pero manteniendo una estructura general en común.

Una clase abstracta funciona como una plantilla base que sirve para definir características y comportamientos generales. En este caso se creó una clase llamada “Empleado”, la cual contiene atributos generales como nombre, edad y salario. Además, incluye métodos que representan acciones que cualquier empleado puede realizar. Sin embargo, esta clase no puede utilizarse directamente para crear objetos porque contiene métodos virtuales puros.

Por otro lado, una interfaz sirve para obligar a las clases a implementar ciertos comportamientos específicos. En esta práctica se utilizó una interfaz llamada “IResponsable”, la cual obliga a que todas las clases derivadas implementen el método generarReporte(). Esto permite que cada tipo de empleado pueda generar reportes relacionados con sus actividades.

También se utilizó polimorfismo, el cual permite trabajar con distintos tipos de objetos utilizando una misma referencia o estructura base. Gracias a esto, el sistema puede manejar diferentes empleados utilizando un solo vector de punteros tipo Empleado, haciendo que cada objeto responda de manera distinta según su tipo.

DESARROLLO DEL SISTEMA

El sistema desarrollado representa una pequeña simulación de una empresa tecnológica que necesita administrar diferentes tipos de empleados. El objetivo principal fue diseñar un programa capaz de manejar distintos trabajadores utilizando una estructura general, pero permitiendo que cada uno tenga comportamientos específicos según su función.

Para comenzar, se creó una clase abstracta llamada "Empleado". Esta clase funciona como base principal del sistema y contiene información general que cualquier trabajador debe tener, como su nombre, edad y salario.

Después se implementó una interfaz llamada "IResponsable". Esta interfaz contiene únicamente el método virtual puro generarReporte().

Posteriormente se desarrollaron tres clases derivadas:
-Programador
-AdministradorRed
-SoporteTecnico

Cada una hereda de la clase Empleado e implementa la interfaz IResponsable.

Uno de los puntos más importantes de la práctica fue el uso de polimorfismo. Para lograrlo se utilizó un vector de punteros tipo Empleado.

CÓDIGO FUENTE COMPLETO

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

class IResponsable
{
public:
    virtual void generarReporte() = 0;
};

class Empleado
{
protected:
    string nombre;
    int edad;
```

```

        double salario;

public:
    Empleado(string n, int e, double s)
    {
        nombre = n;
        edad = e;
        salario = s;
    }

    virtual void trabajar() = 0;

    virtual void realizarActividad()
    {
        trabajar();
    }

    void mostrarInformacion()
    {
        cout << "Nombre: " << nombre << endl;
        cout << "Edad: " << edad << endl;
        cout << "Salario: $" << salario << endl;
    }

    virtual ~Empleado() {}
};

class Programador : public Empleado, public IResponsable
{
public:
    Programador(string n, int e, double s)
        : Empleado(n, e, s) {}

    void trabajar()
    {
        cout << nombre << " esta desarrollando codigo del sistema." << endl;
    }

    void generarReporte()
    {
        cout << "Reporte: avances del sistema completados." << endl;
    }
};

class AdministradorRed : public Empleado, public IResponsable
{
public:
    AdministradorRed(string n, int e, double s)
        : Empleado(n, e, s) {}

    void trabajar()
    {
        cout << nombre << " esta monitoreando servidores y red." << endl;
    }
}

```

```

        void generarReporte()
        {
            cout << "Reporte: estado de la red estable." << endl;
        }
    };

class SoporteTecnico : public Empleado, public IResponsable
{
public:
    SoporteTecnico(string n, int e, double s)
        : Empleado(n, e, s) {}

    void trabajar()
    {
        cout << nombre << " esta atendiendo incidencias tecnicas." << endl;
    }

    void generarReporte()
    {
        cout << "Reporte: tickets atendidos correctamente." << endl;
    }
};

int main()
{
    vector<Empleado*> empleados;

    empleados.push_back(new Programador("Carlos", 25, 12000));
    empleados.push_back(new AdministradorRed("Ana", 30, 15000));
    empleados.push_back(new SoporteTecnico("Luis", 28, 10000));

    for (int i = 0; i < empleados.size(); i++)
    {
        cout << "===== " << endl;

        empleados[i]->mostrarInformacion();
        empleados[i]->trabajar();

        IResponsable* responsable = dynamic_cast<IResponsable*>(empleados[i]);

        if (responsable)
        {
            responsable->generarReporte();
        }
    }

    for (int i = 0; i < empleados.size(); i++)
    {
        delete empleados[i];
    }

    return 0;
}

```

EXPLICACIÓN DEL CÓDIGO

El programa comienza incluyendo las librerías necesarias para trabajar con entrada y salida de datos, cadenas de texto y vectores dinámicos.

Después se crea la interfaz IResponsable. Esta interfaz contiene únicamente el método generarReporte(), el cual es virtual puro.

Posteriormente se desarrolla la clase abstracta Empleado. Esta clase contiene atributos generales como nombre, edad y salario.

Más adelante se crean las clases Programador, AdministradorRed y SoporteTecnico. Estas clases heredan de Empleado e implementan la interfaz IResponsable.

En la función principal se utiliza un vector de punteros tipo Empleado para almacenar diferentes tipos de objetos. Esto permite utilizar polimorfismo.

CAPTURAS DE EJECUCIÓN

Captura 1 – Registro de empleados

```
Nombre: Carlos
Edad: 25
Salario: $12000
Carlos esta desarrollando codigo del sistema.
Reporte: avances del sistema completados.
=====
Nombre: Ana
Edad: 30
Salario: $15000
Ana esta monitoreando servidores y red.
Reporte: estado de la red estable.
=====
Nombre: Luis
Edad: 28
Salario: $10000
Luis esta atendiendo incidencias tecnicas.
Reporte: tickets atendidos correctamente.
```

Captura 2 – Ejecución de tareas

```
Nombre: Carlos
Edad: 25
Salario: $12000
Carlos esta desarrollando codigo del sistema.
Reporte: avances del sistema completados.
=====
Nombre: Ana
Edad: 30
Salario: $15000
Ana esta monitoreando servidores y red.
Reporte: estado de la red estable.
=====
Nombre: Luis
Edad: 28
Salario: $10000
Luis esta atendiendo incidencias tecnicas.
Reporte: tickets atendidos correctamente.
```

Captura 3 – Generación de reportes

```
Nombre: Carlos
Edad: 25
Salario: $12000
Carlos esta desarrollando codigo del sistema.
Reporte: avances del sistema completados.
=====
Nombre: Ana
Edad: 30
Salario: $15000
Ana esta monitoreando servidores y red.
Reporte: estado de la red estable.
=====
Nombre: Luis
Edad: 28
Salario: $10000
Luis esta atendiendo incidencias tecnicas.
Reporte: tickets atendidos correctamente.
```


Captura 4 – Evidencia del polimorfismo

```
Nombre: Carlos
Edad: 25
Salario: $12000
Carlos esta desarrollando codigo del sistema.
Reporte: avances del sistema completados.
=====
Nombre: Ana
Edad: 30
Salario: $15000
Ana esta monitoreando servidores y red.
Reporte: estado de la red estable.
=====
Nombre: Luis
Edad: 28
Salario: $10000
Luis esta atendiendo incidencias tecnicas.
Reporte: tickets atendidos correctamente.
```

CONCLUSIONES

Durante esta práctica se logró comprender de manera más clara el funcionamiento de las clases abstractas, las interfaces y el polimorfismo dentro de la Programación Orientada a Objetos.

También se aprendió cómo organizar mejor un sistema utilizando herencia y reutilización de código. Gracias a esto, el programa puede crecer fácilmente agregando nuevos tipos de empleados sin necesidad de modificar toda la estructura principal.

Una de las principales dificultades fue entender cómo funcionan los métodos virtuales puros y cómo se relacionan las clases derivadas con la clase base abstracta. Sin embargo, al desarrollar el programa completo fue más sencillo comprender la lógica detrás de estos conceptos.

El uso de interfaces también permitió entender cómo obligar a diferentes clases a implementar ciertos comportamientos específicos. Esto hace que el sistema sea más ordenado y profesional.

Finalmente, esta práctica demuestra cómo la Programación Orientada a Objetos puede aplicarse en sistemas reales de empresas para administrar trabajadores, responsabilidades y actividades de manera eficiente, organizada y reutilizable.